JETBOT PROJECT GUIDE #1

Mini Autonomous Vehicle Challenge

Final goal: a fully autonomous JetBot

1. Project Objective and Coding Policy

Build a JetBot that starts from a parking lot, drives through a road environment, handles traffic situations, and safely returns to the designated parking space. The final system must integrate perception, AI, decision making, and control.

Camera → Perception → AI → Decision → Control → Robot

Use the existing JetBot notebooks as building blocks, not as finished solutions.

- Reuse: camera setup, JetBot motor APIs, model-loading code, dataset utilities, and other infrastructure when appropriate.
- Modify or implement: steering logic, speed control, obstacle avoidance, road recovery, intersection decisions, state transitions, cruise control, and parking behavior.
- Do not receive full credit for simply collecting more images, retraining an existing model, or copying notebook cells without changing system behavior.
- For every major modification, be able to explain the problem, your algorithm, the code you changed, and the test result.
- **In every week, you should show the demonstration for each week tasks.**

2. Schedule

Week	Main Development Goal	Required Environment / Facilities
1	**Task 1: Adaptive Road Following** **Task 2: Road Following + Collision Avoidance**	Single lane; straight/gentle/sharp curves; obstacle/pedestrian prop; recovery space
2	**Task 3: Traffic Signs** **Task 4: Intersection Navigation** **Task 5: Parking on the Parking Lot**	STOP/LEFT/RIGHT signs; T/4-way intersection; parking lot and marked parking space
3	**Task 6: State Machine / System Integration**	Combined course: curves, obstacle, signs, intersection, parking
4	**Task 7: Adaptive Speed + Adaptive Cruise Control**	2+ JetBots; straight/curved road; lead vehicle; measurable following gap
5-6	**Task 8: Mini Autonomous Vehicle Challenge / Unknown Testing Course** ➔ **Presentation with Demo**	START/FINISH, parking, curves, signs, intersection, pedestrian/obstacle, changed test conditions

3. Week 1 (Task 1, 2): Road Following and Collision Avoidance

Task 1 — Adaptive Road Following

- Start from the existing Road Following example and make the steering response stable.
- Implement your own control layer between the model output and the motor commands.
- Estimate road curvature or use steering magnitude as a practical curvature indicator.
- Implement curve-aware speed: straight/low curvature → faster; sharp/high curvature → slower.
- Test and document straight, gentle-curve, and sharp-curve sections.

Task 2 — Road Following + Collision Avoidance

- Integrate obstacle detection with normal road following.
- When an obstacle/pedestrian is detected, do more than stop: design an avoidance maneuver and road-recovery procedure.
- Required behavior: ROAD_FOLLOWING → AVOIDANCE → ROAD_RECOVERY → ROAD_FOLLOWING.
- The avoidance and recovery logic must be student-written. The existing Collision Avoidance classifier may be reused as a perception module.

Week 1 checkpoint: demonstrate stable road following, curve-based speed control, obstacle avoidance, and recovery without manual steering.

4. Week 2 (Task 3,4,5): Traffic Signs, Intersections, and Parking

Task 3 — Traffic Sign Recognition

- Collect and label data for STOP, LEFT, RIGHT, and NONE. More images alone are not the objective.

- Use the data to solve a demonstrated recognition problem such as distance, position, or lighting variation.
- The classifier output must be passed to the decision layer; it must not directly command the motors.
- Explain what was changed or improved relative to the example.

Task 4 — Intersection Navigation

- Detect an intersection and combine it with directional information.
- LEFT sign → turn left; RIGHT sign → turn right; otherwise continue straight when appropriate.
- Implement: slow down → enter turn → reacquire road → resume road following.
- The decision and turn-control logic must be student-written.

Task 5 — Parking on the Parking Lot

- The JetBot starts from a designated parking space.
- After completing the route, it must return to the parking lot and stop completely inside the designated parking lines.
- Define a parking approach, alignment procedure, final stopping condition, and failure/retry behavior.
- Parking is a control problem: implement the final alignment and motor-control logic rather than simply stopping near the parking area.

Week 2 checkpoint: demonstrate sign-triggered behavior, intersection turns, and accurate return parking.

5. Week 3:

Task 6: State Machine / System Integration

Combine Tasks 1–5 into one program using an explicit state machine. Minimum states:

- ROAD_FOLLOWING — normal road tracking.
- SLOW_DOWN — reduce speed for a curve or intersection.
- OBSTACLE_AVOIDANCE — avoid an obstacle/pedestrian.
- ROAD_RECOVERY — reacquire the road.
- TURN_LEFT / TURN_RIGHT — execute an instructed turn.
- STOP — respond to a STOP sign or required stop.
- PARKING — approach, align, and stop inside the parking lines.

Each state must specify: entry condition, action/control, exit condition, and priority when multiple events occur.

Important: Do not create a state machine that only calls the old notebooks. The integrated behavior and transitions must be implemented by the team.

6. Week 4:

Task 7: Adaptive Speed + Adaptive Cruise Control

Extend the single-JetBot speed controller to multiple JetBots.

A. Curve-aware speed

- Estimate road curvature from road-following information.
- Set a higher target speed on straight sections and a lower target speed on curves.
- Smooth speed changes to avoid abrupt acceleration/braking.

B. Adaptive cruise control

- Detect the preceding JetBot and estimate the following distance.
- Maintain a target gap instead of using a fixed speed.
- Gap too small → decelerate. Gap too large → accelerate toward the road-based target speed.
- If the lead JetBot stops, the following JetBot must stop while maintaining a safe gap.
- When the lead JetBot moves again, accelerate gradually.

target_speed = min(curve_speed, safe_following_speed)

C. Required tests

- Constant-speed lead vehicle; following vehicle maintains distance.
- Lead vehicle brakes/stops; following vehicle brakes/stops safely.
- Lead vehicle restarts; following vehicle accelerates smoothly.
- Repeat on straight and curved road sections.
- Avoid oscillatory acceleration/braking.

7. Weeks 5–6:

Task 8: Mini Autonomous Vehicle Challenge

Integrate all previous work into one autonomous driving system. The final run begins and ends at the parking lot. No human steering is allowed during the scored run.

Final Course Requirements

- **START/parking lot with clearly marked parking lines.**
- **Straight road, gentle curves, and at least one sharp curve.**
- **At least one T-intersection or four-way intersection.**
- **STOP sign and at least one LEFT/RIGHT directional sign.**
- **Obstacle representing a pedestrian and other road obstruction.**
- **Obstacle avoidance followed by road recovery.**
- **Final return-to-parking section requiring complete parking inside the designated lines.**
- **Multi-JetBot section for adaptive cruise control.**
- **Final test contains changed obstacle/sign positions and/or lighting/road appearance.**
- **Testing on the Unknown Testing Course**

Required Deliverables

- **Final autonomous demonstration, including complete parking at the end.**
- **Make a Presentation Slide which must include:**
  - Every team members should present his/her own slide (job partition is needed)
  - One integrated Python program with separate perception, decision, and control modules.
    - System/state diagram showing information flow and state transitions.
    - The purpose and detail procedures of the Computer Program Codes
  - Experiment record: problem → hypothesis/design → code change → test → result.
    - Training/test data summary explaining why additional data were collected and what problem they solved.
    - You must explain the results with the followings:
      - well summarized Graphs
      - well summarized Tables
- **Submission Guidelines**
  - Submit all materials (source code, report, video, optional data files) as a single ZIP archive.
  - File name should follow this format:
    - “StudentID_Name_mmddyyyy_DataMining.zip”
  - Late submissions will be penalized by 10% per day.